

MACHINE LEARNING SYSTEMS MASTERY

A Roadmap from Mathematical Notation to Production-Grade Research Engineering

"To truly master Machine Learning, treat it as a systems subject. The goal is not 'using libraries,' but understanding how every equation becomes executable code and how papers translate math into tensor operations."

01 Numerical Computing Foundations

Before jumping into ML, you must build the tools. This creates the requisite intuition for gradients, broadcasting, and hardware-level optimization.

Linear Algebra from Scratch

Implement using pure Python first, then optimize with NumPy:

Dot Product

Matrix Multiplication

SVD

Eigen Decomposition

Cosine Similarity

Intuition: Most ML papers are simply creative combinations of these primitives.

Numerical Differentiation & Optimization

- **Gradients:** Implement Forward vs. Central difference to understand $(f(x+h) - f(x)) / h$.
- **Optimizers:** Move beyond SGD. Implement **Momentum**, **RMSProp**, **Adam**, and **AdamW**.

02 The Core Algorithm Suite

Implementation rule: **Strictly NumPy. No Scikit-learn.**

Supervised Learning

- **Linear/Logistic Regression:** Closed-form vs. GD, Cross-entropy, Softmax.
- **SVM:** Kernel trick and Hinge Loss geometry.
- **Trees:** Gini impurity, Information Gain, and Pruning.

Ensembles & Unsupervised

- **Random Forest:** Bootstrap sampling and feature bagging.
- **K-Means++:** Iterative centroid optimization.
- **PCA:** Variance maximization via SVD.

03 Neural Networks: The "Aha!" Moment

This is the single most important transition in your learning journey.

The Mini-PyTorch Project

Build an **Automatic Differentiation Engine** from scratch. This includes:

- **Computational Graph:** Tracking operations for backprop.
- **Tensor Object:** Handling state and gradient accumulation.
- **Activations:** Implement ReLU, GELU, and Swish manually.

Recommended Study: Karpathy's Micrograd or Tinygrad.

04 Modern Architectures

Transition to PyTorch, but reimplement the internal logic of every layer.

- **CNNs:** Manual Conv2D (stride/padding), BatchNorm, and Residual blocks.
- **RNNs/LSTMs:** Backpropagation through time and hidden state management.
- **Transformers:** The "Holy Grail". Build the Multi-head Attention mechanism:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

05 Advanced Generative Modeling

- **VAE:** The Reparameterization Trick and KL Divergence.
- **GANs:** Adversarial training loops (Generator vs. Discriminator).

- **Diffusion:** Forward noise processes and DDPM sampling.

06 The Research Pipeline

Beginner	Word2Vec, LeNet, AlexNet, U-Net
Intermediate	ResNet, Transformer (Attention Is All You Need), BERT
Advanced	ViT, CLIP, Stable Diffusion, LLaMA Architecture

The Final Milestone

You have reached the elite level when you can:

- Translate equations to tensors mentally.
- Predict tensor shapes from diagrams.
- Implement a paper without tutorials.
- Debug gradient flow independently.
- Understand *why* training diverges.